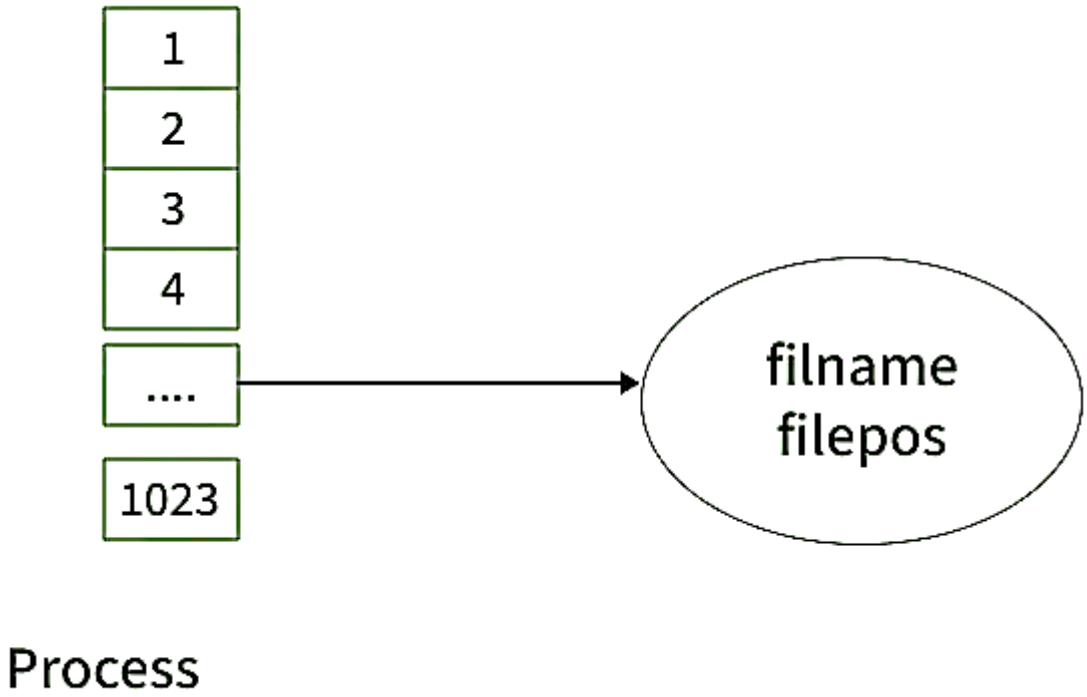
System Call: System calls are the calls that a program makes to the system kernel to provide the services to which the program does not have direct access. In C, all the input and output operations are also performed using input-output system calls. The program makes a call to the system kernel to provide access to input and output devices such as monitors and keyboards.

File Descriptor: A file descriptor is an integer that uniquely identifies an opened file of the process. It is stored in the file descriptor table, in which elements are pointers to file table entries. One unique file descriptor table is provided in the operating system for each process.

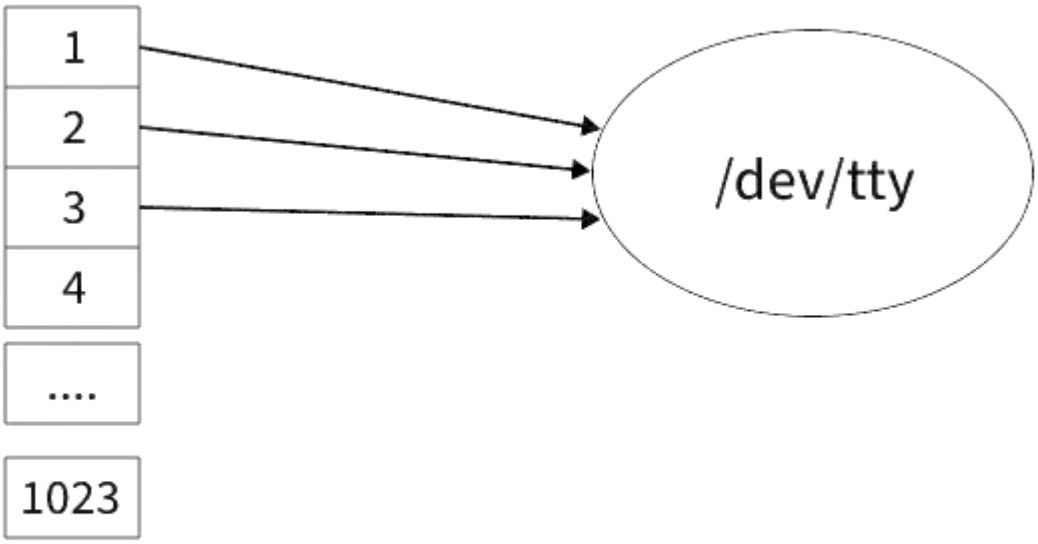
File table entries are a structure in-memory surrogate for an open file, which is created when processing a request to open the file, and these entries maintain file position.



Standard File Descriptors: When any process starts, then that process file descriptors table's fd(file descriptor) 0, 1, 2 open automatically, (By default) each of these 3 fd references file table entry for a file named **/dev/tty**

- **stdin (file descriptor 0):** Standard input stream, whenever we write any character from the keyboard, it reads from stdin through fd 0 and saves to a file named /dev/tty.
- **stdout (file descriptor 1)**: Standard output stream, any output to the video screen, it's from the file named /dev/tty and written to stdout in screen through fd 1.

- **stderr (file descriptor 2)**: We see any error to the video screen, it is also from that file write to stderr in screen through fd 2.



Input-Output System Calls in C: The I/O system calls offer direct access to the operating systems' input and output functionalities and provide complete control over file operations, memory management and device communications. There are 5 input and output system calls in C:

1. Create
2. Open
3. Close
4. Read
5. Write

1. Create: The create system call is used to create a new empty file in C and is provided as create() function. We can specify the permission and the name of the file which we want to create using the create() function. It is defined inside <unistd.h> header file and the flags that are passed as arguments are defined inside <fcntl.h> header file.

Syntax
create (filename, mode);
Parameters

- **filename:** Name of the file which you want to create
- **mode:** Indicates permissions of the new file.

Return Value

- Return first unused file descriptor (generally 3 when because 0, 1, 2 fd are reserved)
- Return -1 when an error occurs.

Example

```
#include <fcntl.h>
#include <unistd.h>
#include <stdio.h>
int main()
{
   // Create a new file or open it
   //if it exists (write-only access)
   // File permissions: rw-r--r--
   int fd = creat("newfile.txt", 0644);

   if (fd == -1) {
      perror("Error creating file");
      return 1;
   }

   // File was successfully created
   printf("File 'newfile.txt' created successfully \n"
         "File descriptor: %d\n", fd);

   // Close the file descriptor
   close(fd);

   return 0;
}
```

Output

```
File 'newfile.txt' created successfully
File descriptor: 3
```

The create() system call works as follows:

- Create a new empty file on the disk.
- Create file table entry.
- Set the first unused file descriptor to point to the file table entry.
- Return file descriptor used, -1 upon failure.

2. Open: The open() function provides the open system call in C. It is used to open the file for reading, writing, or both. It is also capable of creating the file if it does not exist. It is defined inside <unistd.h> header file and the flags that are passed as arguments are defined inside <fcntl.h> header file.

Syntax
open (path, flags);
Parameters

- **Path:** Path to the file which we want to open.
    - Use the absolute path beginning with "/" when you are not working in the same directory as the C source file.
    - Use relative path which is only the file name with extension, when you are working in the same directory as the C source file.
- **flags:** It is used to specify how you want to open the file. We can use the following flags:

Flags	Description
O_RDONLY	Opens the file in read-only mode.
O_WRONLY	Opens the file in write-only mode.
O_RDWR	Opens the file in read and write mode.
O_CREAT	Create a file if it doesn't exist.
O_EXCL	Prevent creation if it already exists.
O_ APPEND	Opens the file and places the cursor at the end of the contents.
O_ASYNC	Enable input and output control by signal.
O_CLOEXEC	Enable close-on-exec mode on the open file.
O_NONBLOCK	Disables blocking of the file opened.
O_TMPFILE	Create an unnamed temporary file at the specified path.

Example

```
#include <errno.h>
#include <fcntl.h>
#include <stdio.h>
#include <unistd.h>

extern int errno;

int main() {
```

```
    // If file does not have in directory
    // then file foo.txt is created.
    int fd = open("foo.txt", O_RDONLY | O_CREAT);
    printf("fd = %d\n", fd);

    if (fd == -1) {

        // Print which type of error have in a code
        printf("Error Number % d\n", errno);

        // print program detail "Success or failure"
        perror("Program");
    }
    return 0;
}
```

Output
fd = 3

The open() system call works as follows:

- Find the existing file on the disk.
- Create file table entry.
- Set the first unused file descriptor to point to the file table entry.
- Return file descriptor used, -1 upon failure.

3. Close: The close system call is provided as close() function in C that tells the operating system that you are done with a file descriptor and closes the file pointed by the file descriptor. It is defined inside <unistd.h> header file.

Syntax
close(fd);
Parameter

- **fd: F**ile descriptor of the file that you want to close.

Return Value

- **0** on success.
- **-1** on error.

Example:

```
#include <fcntl.h>
#include <stdio.h>
```

```
#include <unistd.h>

int main() {
    int fd1 = open("foo.txt", O_RDONLY);
    if (fd1 < 0) {
        perror("c1");
        exit(1);
    }
    printf("opened the fd = % d\n", fd1);

    // Using close system Call
    if (close(fd1) < 0) {
        perror("c1");
        exit(1);
    }
    printf("closed the fd.\n");
}
```

Output

```
opened the fd = 3
closed the fd.
```

Example:

```
#include<stdio.h>
#include<fcntl.h>
int main() {

    // Assume that foo.txt is already created
    int fd1 = open("foo.txt", O_RDONLY, 0);
    close(fd1);

    // assume that baz.tzt is already created
    int fd2 = open("baz.txt", O_RDONLY, 0);

    printf("fd2 = % d\n", fd2);
    exit(0);
}
```

Output

```
fd2 = 3
```

Here, In this code first open() returns 3 because when the main process is created, then fd 0, 1, 2 are already taken by stdin, stdout, and stderr. So

the first unused file descriptor is 3 in the file descriptor table. After that in close() system call is free it these 3 file descriptors and then set 3 file descriptors as null. So when we called the second open(), then the first unused fd is also 3. So, the output of this program is 3.

The close() system call works as follows:

- Destroy file table entry referenced by element fd of the file descriptor table
  - As long as no other process is pointing to it!
- Set element fd of file descriptor table to NULL

4. Read: The read system call, implemented as the read() function, reads the specified number of bytes cnt of input into the memory area indicated by buf from the file indicated by the file descriptor fd. A successful read() updates the access time for the file. The read() function is also defined inside the <unistd.h> header file.

Syntax

read (fd, buf, cnt);

Parameters

- **fd:** file descriptor of the file from which data is to be read.
- **buf:** buffer to read data from.
- **cnt:** length of the buffer.

Return Value

- Return number of bytes read on success.
- Return 0 on reaching the end of file.
- Return -1 on error.
- Return -1 on signal interrupt.

buf needs to point to a valid memory location with a length not smaller than the specified size because of overflow. fd should be a valid file descriptor returned from open() to perform the read operation because if fd is NULL then the read should generate an error. cnt is the requested number of bytes read, while the return value is the actual number of bytes read. Also, sometimes read system call should read fewer bytes than cnt.

Example

```
#include <fcntl.h>
#include <stdio.h>
#include <unistd.h>
```

```
int main() {
   int fd, sz;
   char* c = (char*)calloc(100, sizeof(char));

   fd = open("foo.txt", O_RDONLY);
   if (fd < 0) {
      perror("r1");
      exit(1);
   }

   sz = read(fd, c, 10);
   printf("called read(% d, c, 10). returned that"
         " %d bytes were read.\n",
         fd, sz);
   c[sz] = '\0';
   printf("Those bytes are as follows: % s\n", c);

   return 0;
}
```

Output

called read(3, c, 10). returned that 10 bytes were read.
Those bytes are as follows: 0 0 0 foo.

Suppose that student.txt consists of the 6 ASCII characters “student”. Then what is the output of the following program?

```
#include <fcntl.h>
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int main() {
   char c;
   int fd1 = open("sample.txt", O_RDONLY, 0);
   int fd2 = open("sample.txt", O_RDONLY, 0);
   read(fd1, &c, 1);
   read(fd2, &c, 1);
   printf("c = %c\n", c);
   exit(0);
}
```

Output

c = f
The descriptors *fd1* and *fd2* each have their own open file table entry, so each descriptor has its own file position for *student.txt*. Thus, the read from *fd2* reads the first byte of *student.txt*, and the output is c = f, not c = o.

5. Write: The write system call is provided as write() function. It writes cnt bytes from buf to the file or socket associated with fd. cnt should not be greater than INT_MAX (defined in the limits.h header file). If cnt is zero, write() simply returns 0 without attempting any other action. The write() is also defined inside <unistd.h> header file.

Syntax
write (fd, buf, cnt);
Parameters
- **fd:** file descriptor
- **buf:** buffer to write data from.
- **cnt:** length of the buffer.

Return Value
- Returns the number of bytes written on success.
- Return 0 on reaching the End of File.
- Return -1 on error.
- Return -1 on signal interrupts.

The file needs to be opened for write operations. buf needs to be at least as long as specified by cnt because if buf size is less than the cnt then buf will lead to the overflow condition. cnt is the requested number of bytes to write, while the return value is the actual number of bytes written. This happens when fd has a less number of bytes to write than cnt.

If write() is interrupted by a signal, the effect is one of the following:
- If write() has not written any data yet, it returns -1 and sets errno to EINTR.
- If write() has successfully written some data, it returns the number of bytes it wrote before it was interrupted.

Example:
```
#include<stdio.h>
#include <fcntl.h>

main()  {
```

```
    int sz;

    int fd = open("foo.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);
    if (fd < 0) {
        perror("r1");
        exit(1);
    }

    sz = write(fd, "hello geeks\n", strlen("hello geeks\n"));

    printf("called write(% d, \"hello geeks\\n\", %d)."
           " It returned %d\n", fd, strlen("hello geeks\n"), sz);

    close(fd);
}
```

Output

called write(3, "hello geeks\n", 12). it returned 11

Here, when you see in the file foo.txt after running the code, you get a "*hello geeks*". If foo.txt file already has some content in it then the write a system calls overwrite the content and all previous content is *deleted* and only "*hello geeks*" content will have in the file.

Example:

```
#include <fcntl.h>
#include <stdio.h>
#include <string.h>
#include <unistd.h>

int main(void) {
    int fd[2];
    char buf1[12] = "hello world";
    char buf2[12];

    // assume student.txt is already created
    fd[0] = open("student.txt", O_RDWR);
    fd[1] = open("student.txt", O_RDWR);

    write(fd[0], buf1, strlen(buf1));
    write(1, buf2, read(fd[1], buf2, 12));

    close(fd[0]);
```

```
    close(fd[1]);

    return 0;
}
```

Output

```
hello world
```

Exp-02

Aim: Write programs using the following UNIX operating system calls fork, exec, getpid, exit, wait, close, stat, opendir and readdir

1. fork, getpid, exit, wait, and exec: This program demonstrates creating a child process (fork), identifying processes (getpid), waiting for a child to finish (wait), terminating a process (exit), and replacing a process image with a new program (exec).

```
#include <stdio.h>
#include <stdlib.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <unistd.h>

int main() {
    pid_t pid = fork();

    if (pid == -1) {
        perror("fork failed");
        exit(EXIT_FAILURE);
    }

    if (pid == 0) {
        // Child process
        printf("Child process: PID is %d\n", getpid());
        printf("Child process: Running 'ls -l' using exec...\n");
        // Replace current process image with 'ls -l'
        execlp("ls", "ls", "-l", NULL);
        // execlp only returns if an error occurred
        perror("execlp failed");
        exit(EXIT_FAILURE);
    } else {
        // Parent process
        printf("Parent process: PID is %d, child PID is %d\n", getpid(), pid);
        printf("Parent process: Waiting for child to complete...\n");
        wait(NULL); // Wait for any child process to terminate
        printf("Parent process: Child terminated. Exiting.\n");
        exit(EXIT_SUCCESS);
    }
```

```
    return 0; // Unreachable after exit()
}
```

2. stat: This program demonstrates using stat to retrieve file information (metadata) such as size, permissions, and modification time.

```
#include <stdio.h>
#include <stdlib.h>
#include <sys/stat.h>
#include <time.h>
#include <unistd.h>

int main(int argc, char *argv[]) {
    if (argc != 2) {
        fprintf(stderr, "Usage: %s <filename>\n", argv[0]);
        exit(EXIT_FAILURE);
    }

    struct stat file_stat;

    // Use stat to get file information
    if (stat(argv[1], &file_stat) == -1) {
        perror("stat failed");
        exit(EXIT_FAILURE);
    }

    printf("File: %s\n", argv[1]);
    printf("Size: %ld bytes\n", file_stat.st_size);
    printf("Permissions: %o\n", file_stat.st_mode & 0777);
    printf("Last modified: %s", ctime(&file_stat.st_mtime));

    exit(EXIT_SUCCESS);
}
```

3. opendir and readdir: This program demonstrates opening a directory (opendir) and reading its entries one by one (readdir).

```
#include <dirent.h>
#include <stdio.h>
#include <stdlib.h>
```

```
#include <sys/types.h>
#include <unistd.h>

int main(int argc, char *argv[]) {
    const char *dir_name = (argc > 1) ? argv[1] : ".";
    DIR *dirp;
    struct dirent *dp;

    // Open the directory
    dirp = opendir(dir_name);
    if (dirp == NULL) {
        perror("opendir failed");
        exit(EXIT_FAILURE);
    }

    printf("Contents of directory '%s':\n", dir_name);

    // Read directory entries sequentially
    while ((dp = readdir(dirp)) != NULL) {
        printf("- %s\n", dp->d_name);
    }

    // Close the directory stream
    closedir(dirp); // Note: closedir internally calls close on the underlying file descriptor.

    exit(EXIT_SUCCESS);
}
```

4. close: The close system call is used to terminate a file descriptor. It is implicitly used by functions like fclose() (for standard I/O streams) or closedir(), but it can also be called directly on file descriptors obtained from calls like open(), socket(), or pipe().

```
#include <fcntl.h>
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int main() {
    int fd;
```

```
    const char *filename = "example_file.txt";

    // Open a file (creating it if it doesn't exist)
    fd = open(filename, O_WRONLY | O_CREAT | O_TRUNC, S_IRUSR |
S_IWUSR);
    if (fd == -1) {
        perror("open failed");
        exit(EXIT_FAILURE);
    }

    printf("File opened with file descriptor %d\n", fd);

    // Write some data (optional, just to show file is open)
    if (write(fd, "Hello, world!\n", 14) == -1) {
        perror("write failed");
    }

    // Close the file descriptor
    if (close(fd) == -1) {
        perror("close failed");
        exit(EXIT_FAILURE);
    }

    printf("File descriptor %d closed successfully.\n", fd);

    exit(EXIT_SUCCESS);
}
```